Research Report

PORTFOLIO TASK 1(A)

Unit code: COS40005
Unit Name: Computing Technology Project A
Lecturer: Prof. Dang Tran Khanh
Lecture Date: Saturdays, 13:00 - 17:00
Student Name: Truong Ngoc Gia Hieu
Student ID (VietNam): SWS01217
Student ID (Australia): 105565520
Submission date: Oct 11th, 2026

ACKNOWLEDGEMENT

First and foremost, I would like to express my sincere gratitude to my unit lecturer, Prof. Dang Tran Khanh, for his valuable academic guidance, constructive feedback, and continuous assistance throughout the duration of the Computing Technology Project A unit.

Additionally, I am deeply grateful to the team's industry mentor and client, Mr. Phat (Eric) Pham, for his expert technical insights and practical guidance on establishing our project proposal and project charter prior to moving into the technical phase, all of which significantly contributed to the depth and rigor of this research report.

Furthermore, I declare that the "Research Report - PORTFOLIO TASK 1(A)" is my own work, completed individually in full compliance with Swinburne University of Technology's academic integrity policies and assessment rubrics. In addition, all external research, frameworks, and literature consulted during the preparation of this report have been properly acknowledged and cited using the IEEE referencing standard.

A. Research on the business domain

1.1 Project Discovery and Business Context

In the first two weeks of the course, my team and I conducted an initial investigation to present the project idea, plan, and solution in detail to persuade our client and industry mentor, Mr. Phat (Eric) Pham, through our Project Proposal document [1]. Based on the title, my team detailed that the project focuses on designing and developing an official Slack app bot at the enterprise level to solve the problem of fragmented internal knowledge [1], [3]. Currently, crucial documents are stored across multiple sources and disconnected platforms, leading to employees spending a significant amount of time searching for answers [1], [3]. Consequently, this project is established to query internal knowledge repositories from multiple sources using natural language directly within Slack, receiving instant, accurate answers while serving internal employees securely through a channel-level whitelist mechanism [1], [2], [3].

1.2 Enterprise Domain Operational Challenges

While writing the Project Proposal and Project Charter documents, the team identified three major problems:

- Information fragmentation: Critical technical guides, policies, and API docs are scattered across isolated repositories (e.g., Confluence, GitHub), generating high search friction [1], [3].
- Productivity loss: Navigating outside daily communication tools (Slack) to query external web portals breaks workflow focus, causing employees to waste significant working time searching for information [1], [3].
- Uncontrolled access risks: Traditional search tools lack granular security boundaries, exposing sensitive internal documentation without strict authorization controls [3].

1.3 Market Analysis and Financial Cost Justification

In terms of approaching the solution, the team broke down the topic into core keywords including RAG, multi-source ingestion, and access control [1], [3]. While writing the project proposal paper, all team members suggested a zero-cost model using open-source and free-tier tools [1], [3]. However, after receiving feedback from the client and industry mentor, Mr. Phat (Eric) Pham, the cost model was refined in the project charter to balance zero-cost development with a realistic operational budget [3], [6], [7]:

- Development cost: $0.00 by utilizing open-source libraries, local execution environments, and free-tier developer tools [3], [6].

- Operational cost: Estimated at $3.00 to $5.00 per month under a serverless pay-as-you-go model (supporting approximately 2,000 queries per month via AWS Bedrock and Pinecone Free Tier) [6].

1.4 Identified Business Gap and Value Proposition

By unifying RAG-based multi-source documentation search directly inside Slack, enforcing channel-level whitelist boundaries, and maintaining an ultra-low cost structure, the internal-assistant for enterprise knowledge project directly addresses the gap between fragmented corporate knowledge and secure, instant information retrieval for employees [1], [3], [7].

B. Research on End Users

In the second section of the report, this part aims to identify the target user groups within the enterprise ecosystem and details their distinct operational characteristics, needs, and preferences. Based on accepted documents including the Project Charter and Project Proposal, the team has a deep understanding that addressing user behavior inside corporate communication channels is essential to designing an assistant that minimizes workflow friction while enforcing security boundaries.

2.1 Target User Personas and operational Roles

First, concerning user classification, the primary target audience inside the enterprise environment consists of two main user categories: general technical/operational employees and workspace administrators:

- Specifically, Software Engineers, Product Managers, and Operational Staff: These end users represent the primary consumers of internal documentation. Consequently, their daily workflows require frequent access to technical guides, API documentation, architectural decisions, and standard operating procedures (SOPs) scattered across repositories like Confluence and GitHub. Furthermore, newly onboarded employees also fall into this category, thereby requiring rapid access to foundational project context without continuously interrupting senior team members.
- In contrast, Knowledge Managers and Workspace Administrators: These secondary users oversee internal documentation maintenance and channel access management. In this context, their primary responsibility is ensuring that sensitive internal knowledge is accessible only within authorized channels without imposing complex administrative overhead.

2.2 User Pain Points and Behavioral Preferences

Furthermore, analyzing daily workflow patterns and operational challenges, preliminary research conducted during the project proposal phase identified three primary user pain points:

- First, regarding context-switching fatigue: Employees express a strong preference for obtaining answers directly within their primary communication tool (Slack). Navigating away from ongoing team discussions to search through external web portals or complex Confluence space trees breaks cognitive focus and severely degrades productivity.
- Second, concerning information retrieval latency: Finding verified answers across fragmented repositories currently requires significant manual search time. Users prefer natural language query interfaces that deliver immediate, summarized answers directly inside Slack threads rather than raw keyword search results.
- Finally, with respect to verification and trust requirements: End users exhibit low tolerance for unverified AI-generated responses. Their key preference is receiving synthesized answers accompanied by direct source citations (e.g., clickable links to specific Confluence pages or GitHub README files) to easily verify answer accuracy.

2.3 User Functional and Security Expectations

Consequently, with regard to user functional and security expectations, requirements derived during the project charter phase establish clear operational boundaries:

- Specifically, regarding thread-based conversational interaction: Users expect the bot to respond asynchronously within native Slack threads, preserving channel cleanliness and allowing team members to collaborate around the retrieved information.
- Additionally, concerning transparent access restriction: Users expect clear, non-intrusive feedback when attempting to query the assistant from non-whitelisted channels, ensuring enterprise security policies are transparently enforced without creating user confusion.

C. Research on the solution domain

Building upon the identified user needs and security expectations, this section conducts a systematic domain research on the technical architecture, component paradigms, and framework selections necessary to construct the enterprise assistant. Based on technical evaluations documented in the Project Charter and Project Proposal, the solution domain focuses on establishing a privacy-preserving, zero-cost Retrieval-Augmented Generation (RAG) system deployed locally.

3.1 Architecture Paradigm: Local-First RAG and Cloud APIs

In terms of deployment options, most chat assistants rely on cloud services like OpenAI or Anthropic. However, using cloud services brings two big issues for our project: high monthly costs and security risks from sending private documents over the internet.

- Specifically, regarding data privacy: Keeping company files safe is a top priority. Consequently, running everything on our own machines ensures

that all document data stays on local disks, completely removing any risk of data leaks.
- Furthermore, concerning cost: Our project has a strict zero dollar budget during the proof of concept phase. Therefore, using local open source models gives us good results without paying any cloud API fees.

3.2 Local AI Inference and Embedding Selection

Next, with regard to the model execution runtime, to run open source language models on our own computers, we need a lightweight tool that can handle the workload efficiently:

- First, for the language model: We chose Ollama. Specifically, it works smoothly on different operating systems, offers simple local API endpoints, and runs models like Llama 3 or Mistral very well without needing expensive cloud graphics cards.
- Second, for text embeddings: Which turn text chunks into searchable vectors, we use models like nomic embed text or bge small. In this context, they are accurate and lightweight, allowing them to run alongside the main model easily.

3.3 Vector Database Selection

Furthermore, analyzing vector storage solutions, vector databases act as the core search engine for finding relevant documents:

- In contrast to cloud vector services: Services like Pinecone or Milvus add setup overhead and costs.
- Consequently, we chose ChromaDB: ChromaDB runs directly inside the Python environment and saves data locally on the hard drive. As a result, this avoids managing a separate database server and gives us fast search results for free.

3.4 Communication Interface and Network Connectivity

Finally, connecting Slack to the local application requires a secure setup that does not expose internal network endpoints.

- Specifically, regarding traditional webhooks: Standard Slack HTTP event subscriptions require a public web address, which usually means paying for cloud hosting or using reverse tunneling utilities like Ngrok.
- Alternatively, leveraging Slack Socket Mode: Utilizing Slack Socket Mode establishes a secure connection from the local Python application out to Slack servers using WebSockets. Thereby, the application receives incoming queries over outbound connections without opening inbound public ports or needing a public IP address

D. Kost analysis

First and foremost, mapping the knowledge, skill, and technology requirements of the selected architecture design against the current capabilities of final year students is essential. Specifically, this section evaluates the technical readiness of the team, identifies existing gaps, and outlines a collaborative plan to overcome individual limitations through team support.

4.1 Knowledge and Skill Baseline of Final Year Students

In terms of foundational knowledge and core competencies, final year software engineering students possess a solid academic background.

- Specifically, regarding programming and software concepts: The team brings strong experience in Python development, modular software design, and database fundamentals acquired through coursework and previous university projects.
- Furthermore, concerning collaboration practices: Proficiency in version control workflows and task management ensures structured teamwork and clear role distribution.

4.2 Technology Requirements and Gap Identification

However, analyzing the specific technical stack of the local Retrieval Augmented Generation architecture reveals several individual limitations and gaps.

- First, concerning vector database technologies: While students understand traditional relational databases, managing embedded vector stores like ChromaDB and implementing similarity search algorithms represent an initial technology gap.
- Second, regarding asynchronous communication tools: Building real time event listeners using the Slack Bolt SDK over persistent WebSocket connections requires new practical skills beyond standard web request handling.
- Third, with respect to local AI runtimes: Configuring Ollama and managing resource allocation for local language models on developer workstations introduces a learning curve.

4.3 Team Collaboration and Mitigation Plan

Consequently, to bridge these gaps and overcome individual limitations, the team has established a structured support plan.

- Specifically, regarding peer learning: Team members with stronger backend experience lead knowledge sharing sessions to guide others on asynchronous programming and WebSocket integration.
- Furthermore, regarding task rebalancing: The team distributes development duties based on individual strengths during regular meetings, ensuring that

any technical blocker is resolved collectively without impacting project deadlines.

E. Ethical considerations

In order to ensure legal compliance and maintain user trust, this project adheres to established ethical frameworks and statutory data protection requirements. Specifically, because the enterprise assistant processes internal organizational knowledge and handles user interactions within Slack, system design must strictly comply with Australian privacy laws and international AI governance standards. Consequently, the following details the applicable regulatory frameworks, their core requirements, and the corresponding technical safeguards implemented in this project.

Regulation (Standard)	**Core Requirement**	**Technical Implementation Safeguard**
Australian Privacy Principle 6 (Use or disclosure of personal information)	Personal and sensitive organizational data must not be disclosed or processed outside authorized boundaries without consent.	Operating a strict local first architecture where document processing, vector storage, and model inference occur entirely on local disk storage without external API calls.
Australian Privacy Principle 11 (Security of personal information)	Entities must actively protect retained personal and enterprise information from misuse, loss, and unauthorized access.	Leveraging Slack Socket Mode over outbound WebSockets to avoid opening inbound public ports or exposing static IP endpoints to the public internet.
Privacy Act 1988 (Cth)	Organizations must implement structured controls to handle proprietary and employee data responsibly.	Implementing strict channel level access controls and local database indexing to restrict retrieved search contexts to authorized Slack users.
IEEE Standard for Ethics of Autonomous Systems (P7000)	Autonomous and AI based systems must prioritize data privacy, transparency, and user safety by design.	Enforcing retrieval grounded generation where responses are synthesized strictly from indexed internal files, eliminating unsupported AI fabrications.

References

[1] Office of the Australian Information Commissioner, "Australian Privacy Principles," OAIC, Canberra, ACT, Australia, 2014. [Online]. Available: https://www.oaic.gov.au/privacy/australian-privacy-principles

[2] Commonwealth Parliament of Australia, "Privacy Act 1988," Attorney General Department, Canberra, ACT, Australia, Act No. 119, 1988. [Online]. Available: https://www.legislation.gov.au/Details/C2021C00452

[3] IEEE Standards Association, "IEEE Standard Model Process for Addressing Ethical Concerns during System Design," IEEE Std P7000, 2021. [Online]. Available: https://standards.ieee.org/ieee/7000/6958/

[4] P. Lewis, E. Perez, A. Piktus, Y. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval Augmented Generation for Knowledge Intensive NLP Tasks," in Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.

[5] Ollama Development Team, "Ollama Documentation: Get Up and Running with Llama 3 and Local LLMs," Ollama AI, 2024. [Online]. Available: https://ollama.com/docs

[6] ChromaDB Development Team, "Chroma: The Open Source Embedding Database," ChromaDB, 2024. [Online]. Available: https://docs.trychroma.com/

[7] Slack Technologies, "Slack Bolt for Python Documentation," Salesforce, 2024. [Online]. Available: https://slack.dev/bolt-python/

[8] Slack Technologies, "Understanding Socket Mode in Slack API," Salesforce, 2024. [Online]. Available: https://api.slack.com/apis/connections/socket

[9] LangChain AI, "LangChain Documentation: Building Applications with LLMs through Composability," LangChain, 2024. [Online]. Available: https://python.langchain.com/docs/

[10] LlamaIndex Development Team, "LlamaIndex: Data Framework for LLM Applications," LlamaIndex, 2024. [Online]. Available: https://docs.llamaindex.ai/